

GeoAlgoLab

Interactive Hybrid Closest-Pair-of-Points Algorithm Visualizer

- Dashboard
- Dataset
- Visualizer
- Complexity
- Benchmark
- Threshold Lab
- Methodology
- Report

Algorithm Laboratory

This standalone HTML application demonstrates the three algorithms required for CSA0619: **Brute Force**, **Divide and Conquer**, and the developed **Hybrid approach**. It also measures distance calculations, execution time and the effect of the switching threshold.

How to use: Start with the Dataset tab, generate or upload points, then use the Visualizer and Benchmark tabs.

Input points

100

Closest distance

—

Hybrid threshold

16

Best algorithm

—

Required Algorithms

Approach	Core idea	Expected complexity
Brute Force	Check every possible pair	$O(n^2)$
Divide & Conquer	Split around median x and inspect strip	$O(n \log n)$
Hybrid	D&C for large subproblems, Brute Force below threshold	$O(n \log n)$ asymptotically

GeoAlgoLab

Interactive Hybrid Closest-Pair-of-Points Algorithm Visualizer

Dashboard

Dataset

Visualizer

Complexity

Benchmark

Threshold Lab

Methodology

Report

1. Dataset Explorer

Use generated coordinate data for demonstration or upload a CSV containing **x,y** columns. For a real-world submission, the same interface can accept coordinates from a public geospatial dataset such as airports, weather stations or cell towers.

Number of points

100

Generate Dataset

Upload CSV

Generated 100 coordinate points. Coordinates are normalized demonstration coordinates.



2. Step-by-Step Algorithm Visualizer

Brute Force



Threshold

2

Start

Pause

Next Step

Reset

Step

789/4950

Distance checks

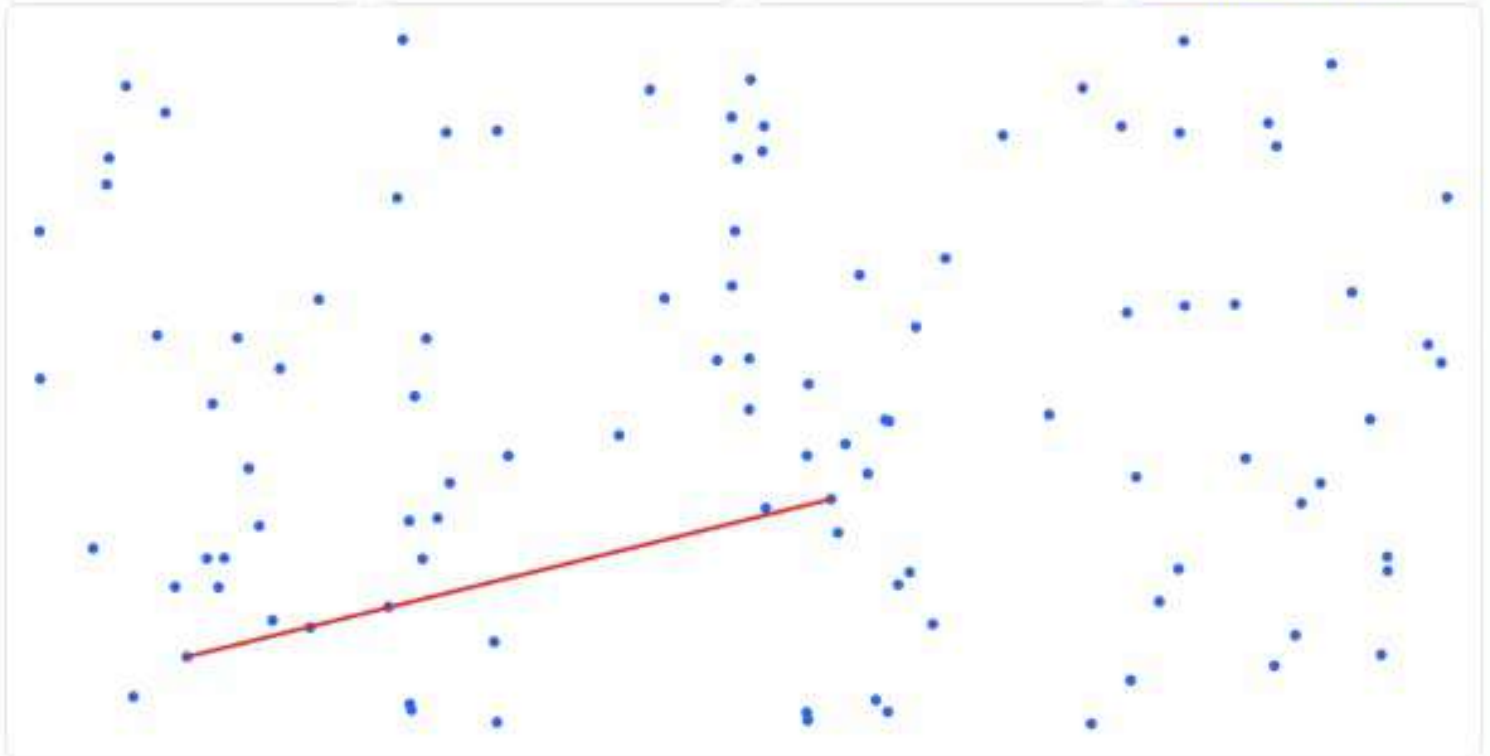
789

Current minimum

—

Phase

Brute Force:
comparing pair



The algorithm is examining point 9 against point 34. The red line represents the current comparison.

3. Complexity & Theory Lab

Brute Force

For n points, the algorithm checks $n(n-1)/2$ pairs.

$T(n) = O(n^2)$, therefore $O(n^2)$ upper bound and $\Omega(n^2)$ lower bound for the standard implementation.

```
for i = 0 to n-1
  for j = i+1 to n-1
    d = distance(P[i], P[j])
    update minimum
```

Divide and Conquer

The points are divided around the median x-coordinate. The two halves are solved recursively and points close to the dividing line are examined in a strip.

$T(n) = 2T(n/2) + O(n)$

Using the Master Theorem: $a=2$, $b=2$, $f(n)=O(n)$. Since $n^{\log_2 2} = n$, this is Case 2, giving $T(n)=O(n \log n)$.

```
closest(P):
  if |P| <= small threshold:
    return bruteForce(P)

  split P at median x
  dl = closest(left)
  dr = closest(right)
  d = min(dl, dr)

  strip = points within d of median
  compare relevant strip neighbors

  return best pair
```

Hybrid Design

```
hybrid(P, threshold):
  if size(P) <= threshold:
    return bruteForce(P)

  split around median x
  solve left recursively
  solve right recursively
  combine using dividing strip
```

Key engineering idea: Brute Force has high asymptotic cost but very small subproblems have low overhead. The Hybrid algorithm exploits this by switching methods at an experimentally selected threshold.

4. Experimental Benchmark

Run all three approaches on increasing input sizes. Results are measured in this browser using the same dataset for each algorithm.

20,40,80,160,320,640

Threshold

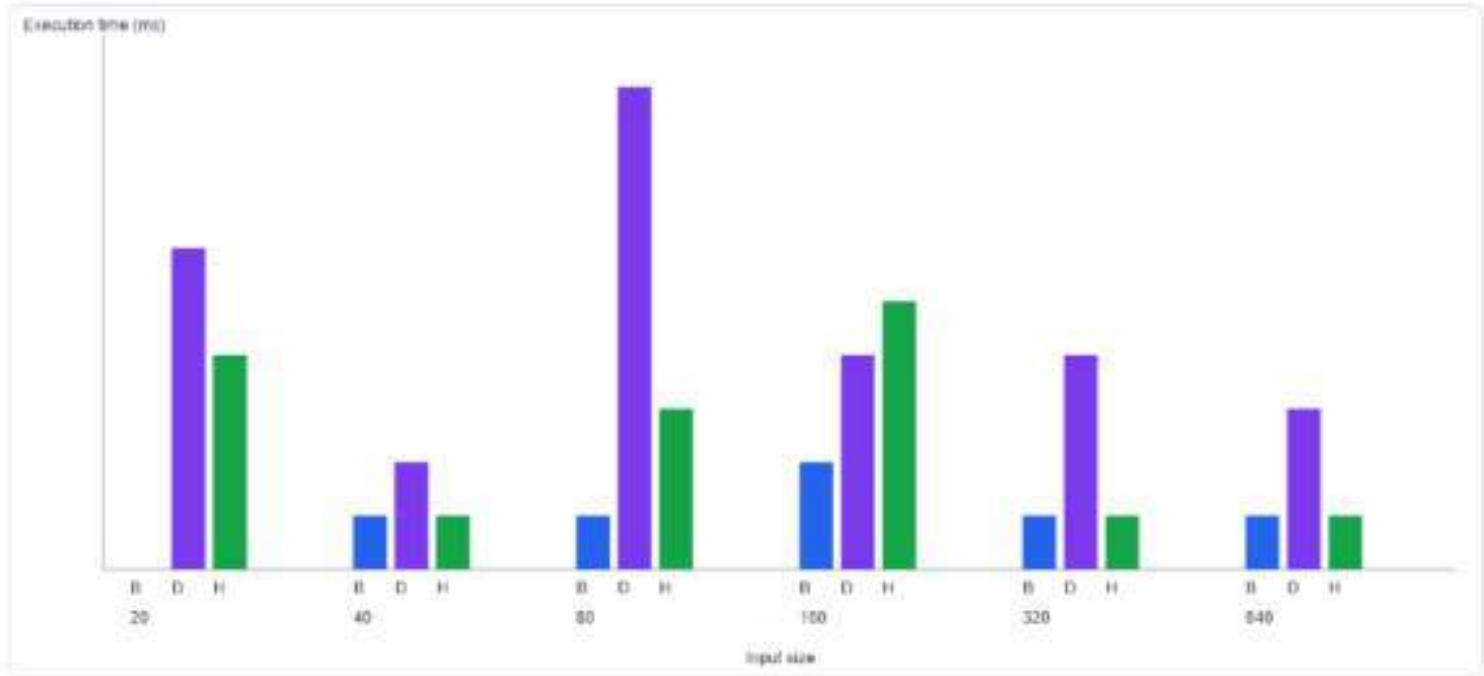
16

Run Benchmark

Benchmark completed using threshold 16. Measurements are from this browser session.

n	Brute time (ms)	D&C time (ms)	Hybrid time (ms)	Brute checks	D&C checks	Hybrid checks
20	0.000	0.600	0.400	190	76	90
40	0.100	0.200	0.100	780	224	184
80	0.100	0.900	0.300	3160	822	384
100	0.200	0.400	0.500	4950	1345	611
100	0.100	0.400	0.100	4950	1345	611
100	0.100	0.300	0.100	4950	1345	611

Performance Chart



5. Hybrid Threshold Optimizer

The threshold should not be chosen arbitrarily. Test several values and select the one producing the lowest measured Hybrid execution time on the target workload.

4,8,12,16,24,32,48,64

Find Best Threshold

Experimentally best threshold: 48
Measured time: 0.000 ms. Use this value as evidence for the hybrid threshold on this workload.

Threshold	Time (ms)	Distance checks
4	0.400	368
8	0.200	366
12	0.200	484
16	0.100	611
24	0.200	611
32	0.100	1214
48	0.000	1214
64	0.100	2453

6. Methodology & Engineering Explanation

Input → Preparation → Algorithms → Threshold → Evaluation



Euclidean Distance

For points $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The implementation compares squared distances internally where possible, avoiding unnecessary square-root operations while preserving the same closest pair.

Correctness

Every algorithm returns the closest pair. The application compares the resulting pair distances and reports whether the implementations agree.

Engineering Trade-off

Brute Force is simple and has low implementation overhead but scales poorly. Divide and Conquer scales much better but introduces recursion, sorting and strip-management overhead. Hybridization attempts to obtain the practical benefits of both.